Enrico Gehring

Principles of Programming

What are the principles of programming

SOLID and KISS principles

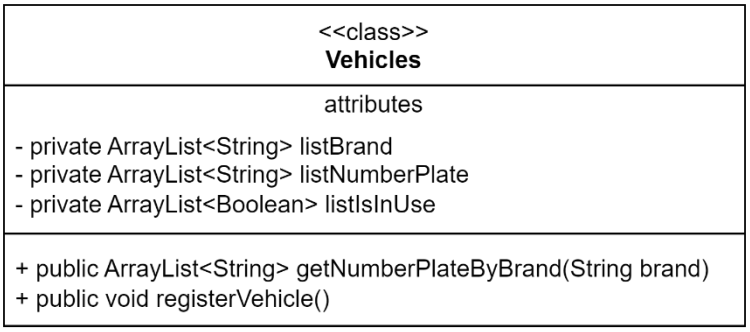
Table of Contents

1. What they are and why

In short, “Principles of Programming” are methods to create clean code which is good to understand and good maintainable. The programming code should be made more structured and cleared by using such principles. A very important component is, that other programmers can understand your source code much better. Another very important component is the maintainability. “A cleanly structured and easy-to-understand source code is an easy-to-maintain source code”. This makes it much easier to extend the source code and add new features.

For example, a good source code is closed to changes and open to extensions. This means that the source code is written in such a way that it does not need to change general code when new features are added. This will ensure that the program runs as it did before the new features were added. This minimizes the time required for programming and testing and saves additional costs in the end. A well programmed source code ensures also a minimized effort by changing features. Only the code that needs to be changed is touched and need to be tested. The rest of the program will remain unaffected, and the changes should have no or minimal impact on functionality.

The following simple example illustrates this. Let's assume that we are a company with a fleet of vehicles. The cars have several characteristics such as brand, license plate and whether it is used. We want to be able to search for each of these properties and list all the vehicles in the fleet. One limitation is that the license plate should be unique. The source code could be structured as follows.



This program has a class in which all vehicles are registered. The “registerVehicle()” method adds all vehicles to its list. There is a list for each property (listBrand, listNumberPlates, listInUse). A vehicle must put its properties in the associated list in the same place. This is done by the "registerVehicle()" method. The following exsample illustrates this.

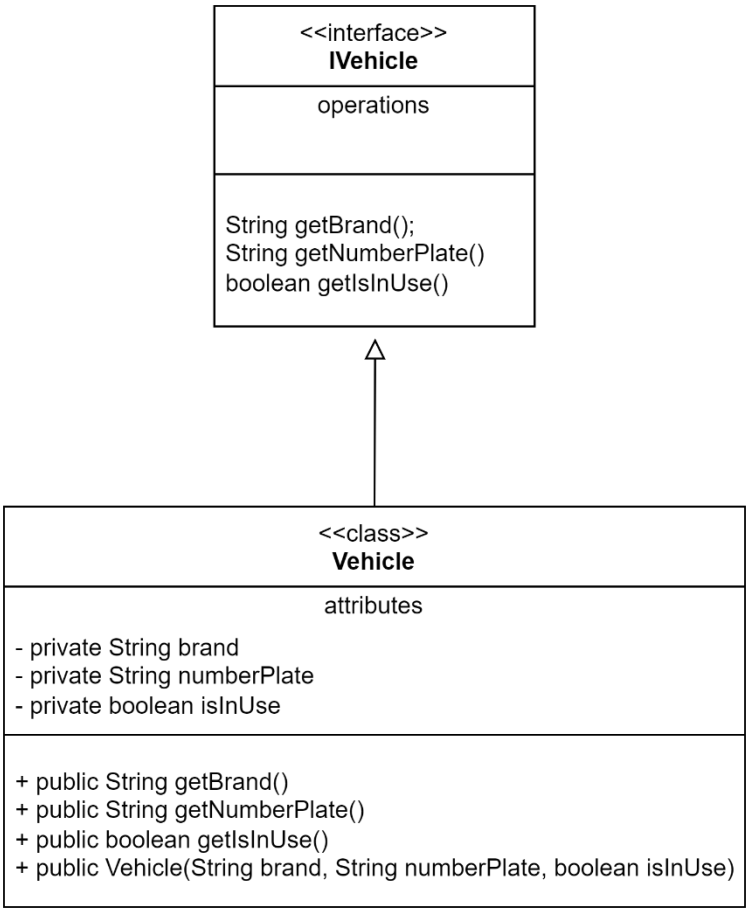
```
public void registerVehicle() {
        this.listBrand.add("Mercedes");
        this.listNumberPlate.add("12AB34");
        this.listIsInUse.add(false);

        this.listBrand.add("BMW");
        this.listNumberPlate.add("43BA21");
        this.listIsInUse.add(true);

        this.listBrand.add("BMW");
        this.listNumberPlate.add("45RE67");
        this.listIsInUse.add(true);

        this.listBrand.add("Volvo");
        this.listNumberPlate.add("34tr56");
        this.listIsInUse.add(true);
    }
```

The function of getting all the brand's license plates is as follows:

```
public ArrayList<String> getNumberPlateByBrand(String brand) {
        int counter = 0;
        ArrayList<String> result = new ArrayList<>();

        for (String b : this.listBrand) {
            if (b.equals(brand)){
                result.add(this.listNumberPlate.get(counter));
            }

            counter++;
        }

        return result;
    }
```

This architecture has some disadvantages. Firstly, it must be ensured that the characteristics of a vehicle are in the same place in every list. If there is only one mistake, the entire program is at risk. Each feature has its own list. Care must be taken when adding or removing a vehicle or, even worse, adding a new property. Remember that each property of a vehicle must be in the same position in the list of affiliations. In addition, a function must be programmed to obtain a list of vehicles by combining the properties.

A much better way to realize that task could look as:



A simple code for the main class might look like this:

```
static final ArrayList<IVehicle> listVehicles = new ArrayList<>();

private static void registerVehicle() {
        listVehicles.add(new Vehicle("Mercedes", "12AB34", false));
        listVehicles.add(new Vehicle("BMW", "43BA21", true));
        listVehicles.add(new Vehicle("BMW", "45RE67", false));
        listVehicles.add(new Vehicle("Volvo", "34tr56", true));
  }

public static void main(String[] args) {
        registerVehicle();
        for (IVehicle vehicle : listVehicles) {
            if (vehicle.getBrand().equals("BMW")) {
                System.out.println();
                System.out.print("BMW with number plate: "
                        + vehicle.getNumberPlate());

                if (vehicle.getIsInUse()) {
                    System.out.print(" -> Is in Use");
                }
            }
        }

        System.out.println();
    }
```

Output:

```
BMW with number plate: 43BA21 -> Is in Use
BMW with number plate: 45RE67
```

This code has many more advantages. There is no need to pay attention to where the data is stored (the same position in the list for each property in the old code). The properties are stored in the class itself and each vehicle has its own instance of the “Vehicle” class. Now

each vehicle remains on its own and is separated from the other vehicles. The possibility of generating errors is significantly reduced.

For example, the list "listVehicles" could be filled with data that is read from the database. It's much easier to handle and maintain a class instead of using the result of a database query directly. Another advantage of the code above is the use of an interface. The main code is now independent of the "Vehicle" class and can easily be extended by another class that inherits from the same interface.

As mentioned earlier, this is a very simple example and should show why “Good Principles” should be used for programming.

2. The KISS – Principle

In this section, you will find some examples of the "KISS" principle. "KISS" stands for "Keep It Stupid Simple". In short, don't overcomplicate it. The following short example should illustrate this.

```
public String[] swap(String[] values) {
        String[] ret = {values[1], values[0]};
        return ret;
    }
```

This function is simple and clear, and it does exactly what it is supposed to do. The task of this function is to swap two values. No more and no less. Many more tasks could be performed, such as testing exactly two elements in the array, testing that the array is not null, and so on. This complicates the function more and more, and that is not the task of this function. Reduce the function as much as possible to its main task. This will make the code much clearer, easier to understand and easier to maintain. If you want to be sure that the “swap” function gets exactly the right number of values to swap, write a function that does that. No more and no less. Such a function could look like this:

```
public String[] checkAndSwap(String[] values) {
        if (null == values) {
            System.out.println("Swap value is null!");
            return null;
        } else if (values.length != 2) {
            System.out.println(
             "Swap value has not the right number of values to swap!");
            return null;
        } else {
            return swap(values);
        }
    }
```

As you can see, the function is supposed to do exactly what it's supposed to do. Nothing more and nothing less. This is the meaning of “Keep It Stupid Simple”.

3. The SOLID – Principles

3.1. Overview

What are the “SOLID - Principles”? The “SOLID - Principles” are a collection of principles.

- ***S*** - *Single – Responsibility – Principle*
- ***O*** - *Open – Close – Principle*
- ***L*** - *Liskov – Substitution – Principle*
- ***I*** - *Interface – Segregation – Principle*
- ***D*** - *Dependency – Inversion – Principle*

3.2. Why the SOLID – Principles?

The "SOLID Principles" are a very powerful tool to design a clean, well-organized and maintainable software architecture. Once you have mastered the "SOLID principles", you will be able to write well-executed code. Remember, your code should be easy to read, highly maintainable, and reusable/replaceable as much as possible.

Depending on your task, you'll use all five principles or just some parts of them. To be able to implement these principles, we use so-called design patterns. Such design patterns could be “Singleton Pattern”, “Factory Design Pattern”, “State Machine Design Pattern” and many more. This topic does not describe design patterns. The "SOLID principles" are described with examples if necessary.

3.3. Single Responsibility Principle

The "Single Responsibility Principle" states that a class is responsible for only one task. A task is a collection of subtasks to solve the task. For example, imagine that you need to program a man-machine interface. One solution might be to write all the functionality in one class. This works, but it is not easy to handle. Now is the time to think about the tasks you need. You may need a user interface (UI), a communication interface (e.g., web services), you may need to store important data (e.g., in a database), and so on. There are now various responsibilities. One responsibility for the user interface, one for the database and one for communication. Let's assume that a menu is needed for the user interface. Menu management could be outsourced by a separate task that is responsible for managing all menu tasks in one place. Each of these tasks is represented by their own class. The interaction between these tasks takes place via interfaces. Now it's easy to replace tasks with other tasks. Your job as a programmer is to identify all the program's tasks, break them down into subtasks that stand on their own, and create a good interface to communicate between the tasks.

3.4. Open Close Principle

What does “Open – Close" mean? A software (functions, classes, etc.) should be “Open to extensions and closed to modifications”. Let's go into detail with the help of an example.

Let's say you want to calculate the area of a square. The formula for calculating the square is $A = x^2$. We create a class “Square” that gets the length of its lines (x). To simplify the example, let's set all variables to public.

```
class Square {
    public int line_x;
}
```

Let's further assume that we want to be able to calculate the area of a single shape and all shapes. We will define a class that provides all the necessary functionalities to do this.

```
class CalculateArea {
    public int calculateAreaOfSquare(Square s) {
        return s.line_x * s.line_x;
    }
    public int calculateAreaOfAllShapes (
                        ArrayList<Square> listSquares) {
        int result = 0;
        for (Square s : listSquares) {
            result += this. calculateAreaOfSquare (s);
        }
        return result;
    }
}
```

Now we have three squares with a side length of 3, 8 and 10. The main function could be as follows:

Output:

Area of Square 1: 9

Area of Square 2: 64

Area of Square 3: 100

Area of all Squares: 173

```
public static void main(String[] args) {
        Square s1 = new Square();
        s1.line_x = 3;

        Square s2 = new Square();
        s2.line_x = 8;

        Square s3 = new Square();
        s3.line_x = 10;

        ArrayList<Square> listSquares = new ArrayList<>();
        listSquares.add(s1);
        listSquares.add(s2);
        listSquares.add(s3);

        CalculateArea c = new CalculateArea();
        int areaSquare_1 = c.calculateAreaOfSquare(s1);
        int areaSquare_2 = c. calculateAreaOfSquare (s2);
        int areaSquare_3 = c. calculateAreaOfSquare (s3);
        int areaAllSquares = c.calculateAreaOfAllShapes(listSquares);

        System.out.println("Area of Square 1: "+ areaSquare_1);
        System.out.println("Area of Square 2: "+ areaSquare_2);
        System.out.println("Area of Square 3: "+ areaSquare_3);
        System.out.println("Area of all Squares: "+ areaAllSquares);
}
```

So far so good. Now let's assume, there is needed a new shape. A rectangle. The program needs to be extended by a new class "Rectangle". That class has two parameters for line_x and line_y. The formula for calculating the area of the rectangle is $A = ab$.

```
class Rectangle {
    public int line_x;
    public int line_y;
}
```

The "CalculateArea" class must be modified to calculate the area of a rectangle and the area of all shapes. The class now looks like this:

```
class CalculateArea {
    public int calculateAreaOfSquare(Square s) {
        return s.line_x * s.line_x;
    }
    public int calculateAreaOfRectangle(Rectangle r) {
        return r.line_x * r.line_y;
    }
    public int calculateAreaOfAllShapes (
            ArrayList<Square> listSquares,
            ArrayList<Rectangle> listRectangles){
        int result = 0;
        for (Square s : listSquares) {
            result += this.calculateAreaOfSquare(s);
        }
        for (Rectangle r : listRectangles) {
            result += this.calculateAreaOfRectangle(r);
        }
        return result;
    }
}
```

Let's say we want to calculate the area of three rectangles with side lengths of (2, 3), (10, 20), and (5, 4) in addition to the squares.

```
public static void main(String[] args) {
        …… Code from Square
      Rectangle r1 = new Rectangle();
      r1.line_x = 2;
      r1.line_y = 3;

      Rectangle r2 = new Rectangle();
      r2.line_x = 10;
      r2.line_y = 20;

      Rectangle r3 = new Rectangle();
      r3.line_x = 5;
      r3.line_y = 4;

      listRectangles.add(r1);
      listRectangles.add(r2);
      listRectangles.add(r3);

      int areaRectangle_1 = c.calculateAreaOfRectangle(r1);
      int areaRectangle_2 = c. calculateAreaOfRectangle (r2);
      int areaRectangle_3 = c. calculateAreaOfRectangle (r3);
      int areaOfAllShapes = c.calculateAreaOfAllShapes(
                                    listSquares, listRectangles);

      System.out.println();
      System.out.println("Area of Rectangle 1: "+ areaRectangle_1);
      System.out.println("Area of Rectangle 2: "+ areaRectangle_2);
      System.out.println("Area of Rectangle 3: "+ areaRectangle_3);
      System.out.println("Area of all Shapes: "+ areaOfAllShapes);
}
```

Output:

Area of Square 1: 9

Area of Square 2: 64

Area of Square 3: 100

Area of all Squares: 173

Area of Rectangle 1: 6

Area of Rectangle 2: 200

Area of Rectangle 3: 20

Area of all Shapes: 399

This code violates the “Open Clos Principle”. Open for extension and closed for modifications. The reason for this is that we had to modify the “CalculateArea” class when we added the new “Rectangle” class. Let's say that we've added more and more shapes over time. Shapes like Circle, Polygon, Trapezoid, and more. It always needs to modify the “CalculateArea” class when we add a new shape. This gives a lot of room for errors and the class "CalculateArea" is getting bigger and bigger. Think about the benefits of good programming. That's clean code. Good to understand and maintain.

Let's see how we can do better. First, we're going to create an interface. Every shape must implement this interface. The “CalculateArea” class uses this interface to calculate the area of a single shape or all shapes. The “CalculateArea” class is now free of each individual class. It no longer cares about which shape area must be computed. The calculation of the area is done in the shape class itself. You can add as many shapes as you want without modifying the “CalculateArea” class. That means “Close for Modification”. The big

advantage is that you no longer must worry about whether the calculation of the other shape still works. The reason for this is that you haven't made any modifications. You've just added a new shape

```
interface IShape {
    int calculateArea();
}

class Square_2 implements IShape {
    private final int line_x;

    public Square_2(int line_x) {
        this.line_x = line_x;
    }

    @Override
    public int calculateArea() {
        return this.line_x * this.line_x;
    }
}

class Rectangle_2 implements IShape {
    private int line_x;
    private int line_y;

    public Rectangle_2(int line_x, int line_y) {
        this.line_x = line_x;
        this.line_y = line_y;
    }

    @Override
    public int calculateArea() {
        return this.line_x * this.line_y;
    }
}
```

(extension). The program could now look like this:

```
class CalculateArea_2 {
    public int calculateShape(IShape shape) {
        return shape.calculateArea();
    }

    public int calculateAreaOfAllShapes(
                                    ArrayList<IShape> listShapes) {
        int result = 0;

        for (IShape shape : listShapes) {
            result += shape.calculateArea();
        }

        return result;
    }
}
```

The main function could look like:

```
Square_2 square_1 = new Square_2(3);
Square_2 square_2 = new Square_2(8);
Square_2 square_3 = new Square_2(10);

Rectangle_2 rect_1 = new Rectangle_2(2, 3);
Rectangle_2 rect_2 = new Rectangle_2(10, 20);
Rectangle_2 rect_3 = new Rectangle_2(5, 4);

ArrayList<IShape> listShapes = new ArrayList<>();
listShapes.add(square_1);
listShapes.add(square_2);
listShapes.add(square_3);
listShapes.add(rect_1);
listShapes.add(rect_2);
listShapes.add(rect_3);

CalculateArea_2 calcArea = new CalculateArea_2();
int resSquare_1 = calcArea.calculateShape(square_1);
int resSquare_2 = calcArea.calculateShape(square_2);
int resSquare_3 = calcArea.calculateShape(square_3);
int resRect_1 = calcArea.calculateShape(rect_1);
int resRect_2 = calcArea.calculateShape(rect_2);
int resRect_3 = calcArea.calculateShape(rect_3);
int resAllShapes = calcArea.calculateAreaOfAllShapes(listShapes);
```

The calculation is decoupled from a single class. Now it is possible to extend the program without changing the calculation class "ComputeArea". Suppose we need to extend the program to calculate the area of a circle. All you must do is add a new class "Circle" and include it in the main function.

```
class Circle implements IShape {
        private final int radius;

        public Circle(int radius) {
                this.radius = radius;
        }

        @Override
        public int calculateArea() {
                return (int) Math.round(3.14 * this.radius);
        }
}
```

Including in the main class:

```
…
        Circle circle = new Circle(10);
        int resCircle = calcArea.calculateShape(circle);
        listShapes.add(circle);
        resAllShapes = calcArea.calculateAreaOfAllShapes(listShapes);
```

As you can see, we have not changed any existing code. We have only added new code. This means that we have extended our program without changing the existing code. This ensures that the existing code still runs exactly as it did before the new shape class was added.

3.5. Liskov – Substitution – Principle (LSP)

"LSP is based on the concept of substitutability."

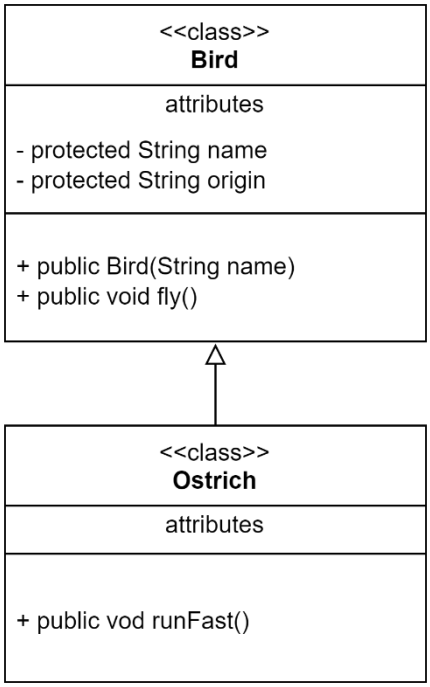
What does it mean? It can be replaced by an object of a subclass without the program aborting.

What does breaking mean? Breaking means violating the rules of the program. By replacing an object of a subclass, the program could stop or, if it is still running, lead to an incorrect result.

Again. The LSP states that there is a class A and a class B. Class B is a subclass of class A. Now you can replace an object of class A with an object of class B without destroying the program.

Let us consider the following example:

Suppose we have a class bird. Let us further assume that a bird has a name, an origin and can fly. Now we have an ostrich. An ostrich is a bird and is inherited from the class Bird. The ostrich has the property of running fast. The class structure could look like this:



Now we will generate two instances:

```
Bird falcon = new Bird();
Ostrich paul = new Ostrich();
```

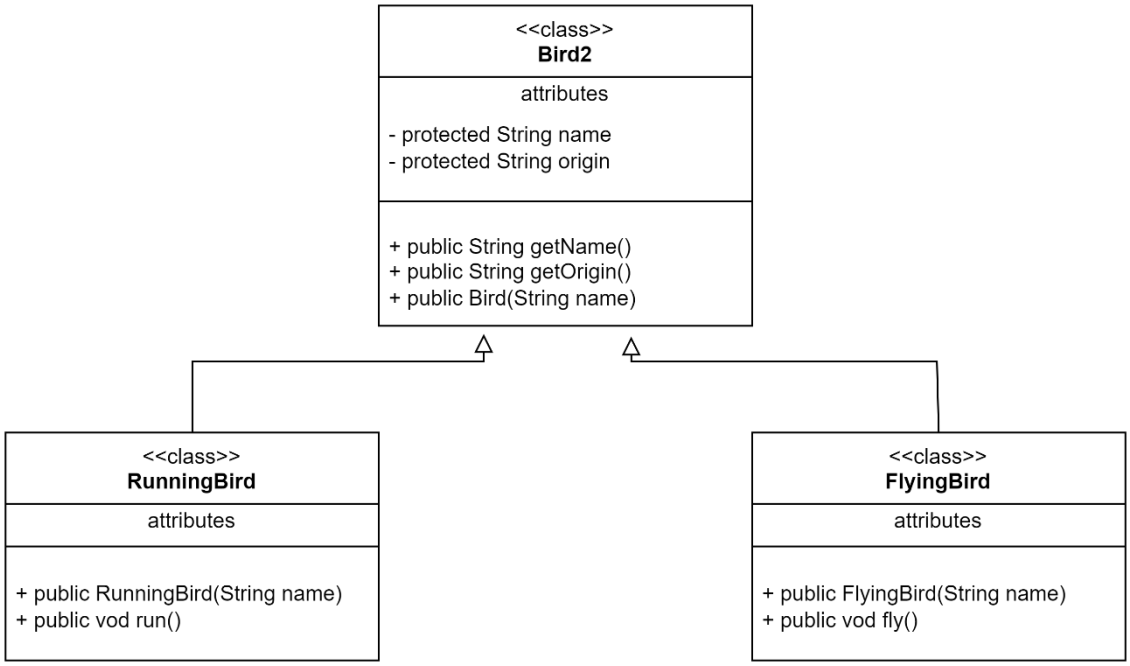
As you can see, we have a method called *fly()* in the parent class *Bird*. This method can be called by any subclass. LSP says that it can replace an object a with an object b (subclass) without breaking the program. In our case, it would look like this:

```
falcon.fly(); // that's correct
paul.fly(); // that's wrong
```

What has happened? The program is not stopped. Both instances can call the fly method. The thing is that an ostrich cannot fly. This means

that the Ostrich class should not be able to call the fly method. The program, as it is, breaks LSP.

Now we are doing it in a better way. We still have the parent class *Bird*. Now we will add two more classes that will be inherited from the Bird class. The classes are *RunningBirds* and *FlyingBirds*.



Change the main program to:

```
Bird2 sparrow = new Bird2("Sparrow");
FlyingBird falcon2 = new FlyingBird("Falcon");
RunningBird ostrich = new RunningBird("Ostrich");

sparrow.getName(); // Output: “Sparrow”
falcon2.fly(); // that’s correct.
ostrich.run(); // that’s correct.
falcon2.getName(); // that’s correct
ostrich.getName(); // that’s correct
```

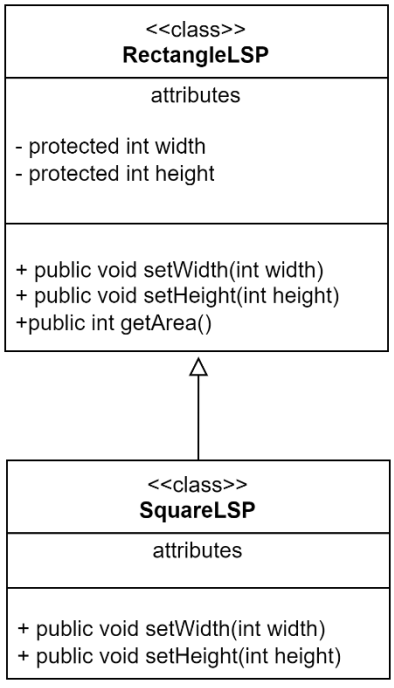
Our program is now LSP-compliant. The instance of the parent class Bird2 can be replaced by an inherited class without violating LSP. Each specialized bird has its own class that inherits from the parent class Bird2.

There are several ways to solve the LSP problem. One, as shown, is through specialized subclasses. Another way could be through interfaces.

To find out how you can make your code LSP-compliant, you need to see which properties of a bird are common to all birds and which are specialized. All properties that birds have in common are placed in the parent class.

Let's show this with another example.

Let's assume that we want to calculate the area of two shapes, as described in section 3.3. The shapes in our example are a rectangle and a square. A square is a special form of a rectangle in which all sides are of the same length.



```
class RectangleLSP {
	protected int width;
	protected int height;

	public void setHeight(int height) {
		this.height = height;
	}

	public void setWidth(int width) {
		this.width = width;
	}

	public int getArea() {
		return this.width * this.height;
	}
}

class SquareLSP extends RectangleLSP {
	@Override
	public void setWidth(int width) {
		super.width = width;
		super.height = width;
	}

	@Override
	public void setHeight(int height) {
		super.width = height;
		super.height = height;
	}
}
```

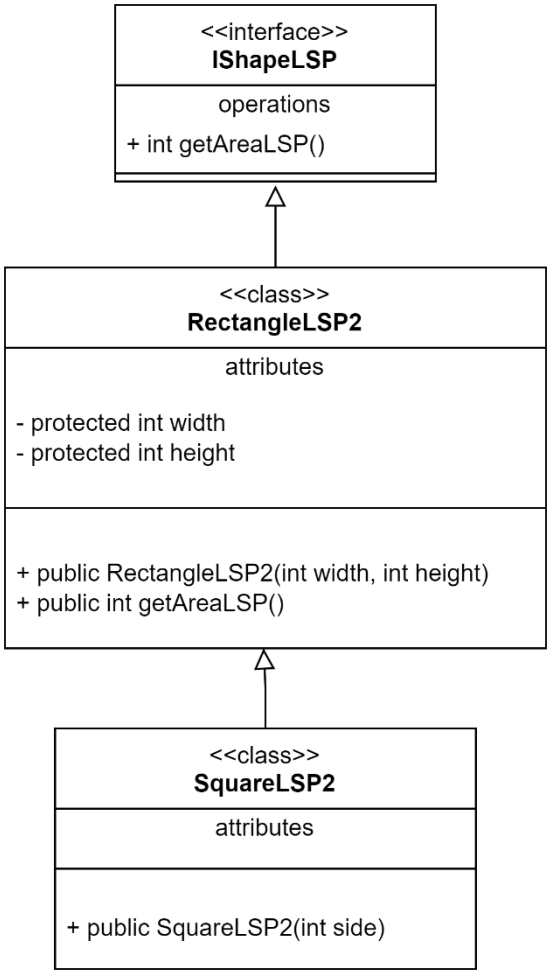
Now let's assume that we have a method to print the area by receiving an instance of a *RectangleLSP* as a parameter.

```
private static void printAreaLSP(RectangleLSP rectangle) {
        rectangle.setHeight(2);
        rectangle.setWidth(5);

        System.out.print("The area of the Rectangle is: " + rectangle.getArea());
}

RectangleLSP r = new RectangleLSP();
SquareLSP s = new SquareLSP();
```

LSP says that we can replace the instance r with the instance s and expect a valid result. The printAreaLSP method does not care what type of rectangle it receives. We expect 10 as the result. Let's take a look at what happens when we call the printAreaLSP method with instance r and instance s.

```
printAreaLSP(r); // As expected.
printAreaLSP(s); // Not as ecpected.
```

How can we fix it? We can fix it by using an interface. The class model looks like this:




```
interface IShapeLSP {
    int getAreaLSP();
}
```

The RectangleLSP and SquareLSP classes are changed as follows:

```
class RectangleLSP2 implements IShapeLSP {
    protected int width;
    protected int height;

    public RectangleLSP2(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getAreaLSP() {
        return this.width * this.height;
    }
}

class SquareLSP2 extends RectangleLSP2 {

    public SquareLSP2(int side) {
        super(side, side);
    }
}
```

The method printAreaLSP is changed as follows:

```
public void printAreaLSP2(IShapeLSP shape) {
        System.out.print("The area of the Rectangle is: " + shape.getAreaLSP());
}
RectangleLSP2 rLSP = new RectangleLSP2(2, 5);
SquareLSP2 sLSP = new SquareLSP2(5);

printAreaLSP2(rLSP); // Expect 10 and get out 10. Valid Result.
printAreaLSP2(sLSP); // Expect 25 and get out 25. Valid Result.
```

3.6. Interface – Segregation – Principle (ISP)

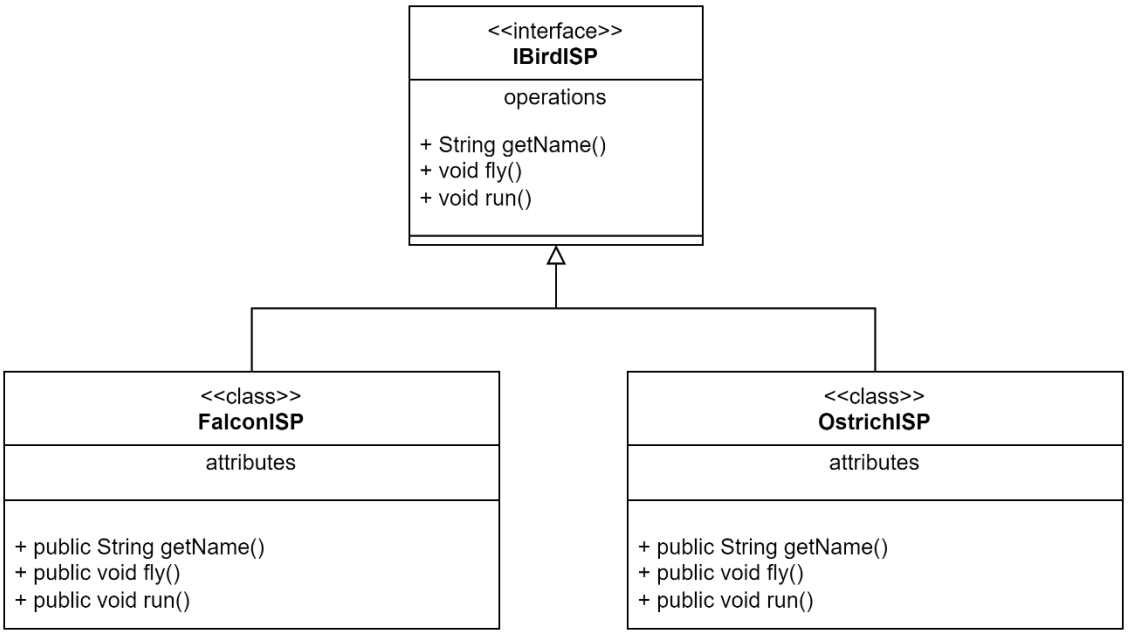
"The ISP states that no client should be forced to depend on methods or functions that they do not use."

What does it mean? Suppose we have an interface with several methods/functions. This interface is implemented by several classes. Some of these classes need all the methods/functions of the interface and some classes only need some of them.

All classes must implement all methods/functions of the interface. It does not matter whether they need the methods/functions or not. The body of the methods/functions that are not required is empty as they are not used. **That's breaking the ISP.**

Let's illustrate this with an example.

Let us assume we have birds. Those birds are represented by an interface with the following properties: Name, Fly, Run.



As you can see, all classes that implement the "IBirdISP" interface are forced to implement all methods/functions. The "Falcon" class does not

require the “run()” method and the “Ostrich” class does not require the “fly” method.

The source code looks like this:

```
interface BirdISP {
    public String getName();
    public void fly();
    public void run();
}

class FalconISP implements BirdISP {
    @Override
    public String getName() {
        return "Falcon";
    }

    @Override
    public void fly() {
        System.out.println("I'am a flying bird!");
    }

    @Override
    public void run() {
    }
}

class OstrichISP implements BirdISP {
    @Override
    public String getName() {
        return "Ostrich";
    }

    @Override
    public void fly() {
    }

    @Override
    public void run() {
        System.out.println("I'am a running Bird!");
    }
}
```

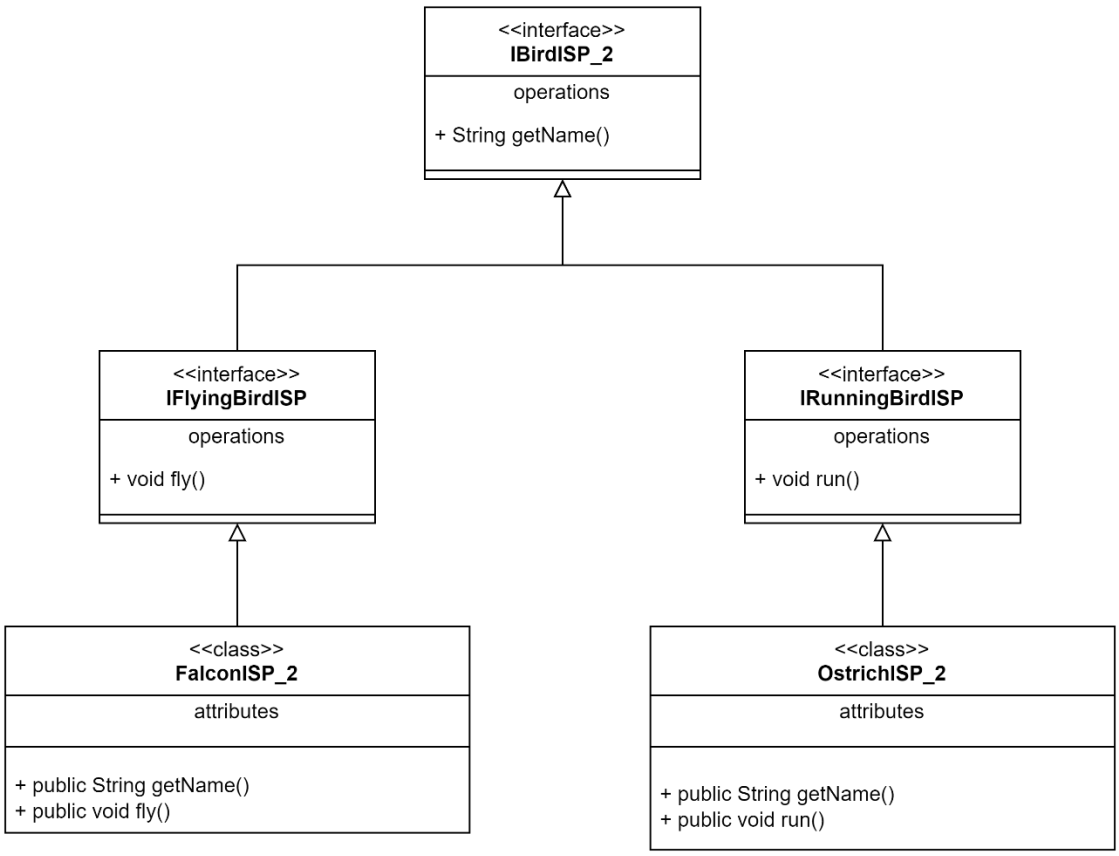
What can we do to adapt to the ISP?

First determine which methods/functions are required by all subclasses. In our case, this is the "getName()" function. Then determine the methods/functions that several classes have in common. Each of these identified collections is given its own interface. A class now implements this interface that is required.

Let's do it for our example:

Method/Function	Belonging Class
getName()	Falcon, Ostrich (all classes)
fly()	Falcon
Run()	Ostrich

Let's change the code:



```
interface IBirdISP_2 {
    public String getName();
}

interface IFlyingBirdISP extends IBirdISP_2 {
    public void fly();
}

interface IRunningBirdISP extends IBirdISP_2 {
    public void run();
}

class FalconISP_2 implements IFlyingBirdISP {

    @Override
    public void fly() {
        System.out.println("I'am a flying bird!");
    }

    @Override
    public String getName() {
        return "Falcon";
    }
}

class OstrichISP_2 implements IRunningBirdISP {

    @Override
    public void run() {
        System.out.println("I'am a running Bird!");
    }

    @Override
    public String getName() {
        return "Ostrich";
    }
}
```

As you can see, each class only implements the methods/functions that it needs.

3.7. Dependency – Inversion – Principle (DIP)

"The DIP specifies an interface abstraction between higher-level components and lower-level components. In short, DIP states that high-level software components are not dependent on low-level software components.

What are high level components and low level components?

High-level components are classes that use low-level components (instances of the class). The high-level components depend on the low-level components. This is because they use them. There can be several levels of dependency. One of them is direct dependency through the use of an instance of the low-level component. Let's show it with an abstract example.

```
class LowLevelComponent {
    public void doSomething() {
        System.out.println("The Low-Level Component is doing something!");
    }
}

class HighLevelComponent {
    public void run(LowLevelComponent l) {
        l.doSomething();
    }
}
```

As you can see, the “HighLevelComponent” class uses the “LowLevelComponent” class as an instance parameter in the “run(LowLevelComponent l)” method. It is directly dependent on the “LowLevelComponent” class. This violates the DIP. Firstly, you can only use the “LowLevelComponent” class in the “run(LowLevelComponent l)” method and secondly, if you remove the “LowLevelComponent” class, you must also remove it from the “run(LowLevelComponent l)” method in the “HighLevelComponent” class.

How can dependencies be reversed? Through interfaces.

```
interface IDoSomething {
    public void doSomething();
}

class LowLevelComponent_2 implements IDoSomething {

    @Override
    public void doSomething() {
        System.out.println("The Low-Level Component is doing something!");
    }
}

class HighLevelComponent_2 {
    public void run(IDoSomething i) {
        i.doSomething();
    }
}
```

As you can see, the “High-Level Component” class is no longer dependent on the “Low-Level Component” class. Another advantage is that you can use any class as a “Low-Level Component” class that implements the “IDoSomething” interface. You can also remove the entire “Low-Level Component” class without having to change the “run(...)” method in the “High-Level Component” class.

4. Complete Sample Program

```
package principlesofprogramming;

import java.util.ArrayList;


class Vehicles {
    private final ArrayList<String> listBrand;
    private final ArrayList<String> listNumberPlate;
    private final ArrayList<Boolean> listIsInUse;

    public Vehicles() {
        this.listBrand = new ArrayList<>();
        this.listNumberPlate = new ArrayList<>();
        this.listIsInUse = new ArrayList<>();
    }

    public ArrayList<String> getNumberPlateByBrand(String brand) {
        int counter = 0;
        ArrayList<String> result = new ArrayList<>();

        for (String b : this.listBrand) {
            if (b.equals(brand)){
                result.add(this.listNumberPlate.get(counter));
            }

            counter++;
        }

        return result;
    }

    public void registerVehicle() {
        this.listBrand.add("Mercedes");
        this.listNumberPlate.add("12AB34");
        this.listIsInUse.add(false);

        this.listBrand.add("BMW");
        this.listNumberPlate.add("43BA21");
        this.listIsInUse.add(true);

        this.listBrand.add("BMW");
        this.listNumberPlate.add("45RE67");
        this.listIsInUse.add(true);

        this.listBrand.add("Volvo");
```

```
        this.listNumberPlate.add("34tr56");
        this.listIsInUse.add(true);
    }
}

/*
Vehicle Example
*/
interface IVehicle {
    String getBrand();
    String getNumberPlate();
    boolean getIsInUse();
}

class Vehicle implements IVehicle{
    private String brand = "";
    private String numberPlate = "";
    private boolean isInUse = false;

    public Vehicle(String brand, String numberPlate, boolean isInUse) {
        this.brand = brand;
        this.numberPlate = numberPlate;
        this.isInUse = isInUse;
    }

    @Override
    public String getBrand() {
        return this.brand;
    }

    @Override
    public String getNumberPlate() {
        return this.numberPlate;
    }

    @Override
    public boolean getIsInUse() {
        return this.isInUse;
    }
}

/*
Open Close Principle
Bad Example
*/
class Square {
    public int line_x;
}
```

```
class Rectangle {
    public int line_x;
    public int line_y;
}

class CalculateArea {
    public int calculateAreaOfSquare(Square s) {
        return s.line_x * s.line_x;
    }

    public int calculateAreaOfRectangle(Rectangle r) {
        return r.line_x * r.line_y;
    }

    /*
     * When I have to add a new shape, I have also to modify the function.
     * Thats again the Open - Close - Principle.
     */
    public int calculateAreaOfAllShapes (
            ArrayList<Square> listSquares,
            ArrayList<Rectangle> listRectangles){
        int result = 0;

        for (Square s : listSquares) {
            result += this.calculateAreaOfSquare(s);
        }

        for (Rectangle r : listRectangles) {
            result += this.calculateAreaOfRectangle(r);
        }

        return result;
    }
}

/*
Open Close Principle
Good Example
*/
interface IShape {
    int calculateArea();
}

class Square_2 implements IShape {
    private final int line_x;

    public Square_2(int line_x) {
```

```
        this.line_x = line_x;
    }

    @Override
    public int calculateArea() {
        return this.line_x * this.line_x;
    }
}

class Rectangle_2 implements IShape {
    private final int line_x;
    private final int line_y;

    public Rectangle_2(int line_x, int line_y) {
        this.line_x = line_x;
        this.line_y = line_y;
    }

    @Override
    public int calculateArea() {
        return this.line_x * this.line_y;
    }
}

class Circle implements IShape {
    private final int radius;

    public Circle(int radius) {
        this.radius = radius;
    }

    @Override
    public int calculateArea() {
        return (int) Math.round(3.14 * this.radius);
    }
}

class CalculateArea_2 {
    public int calculateShape(IShape shape) {
        return shape.calculateArea();
    }

    public int calculateAreaOfAllShapes(ArrayList<IShape> listShapes) {
        int result = 0;

        for (IShape shape : listShapes) {
            result += shape.calculateArea();
        }
```

```
        return result;
    }
}

/*
The Liskov - Substitution - Principle (LSP)
*/
class Bird {
    protected String name;
    protected String origin;

    public Bird(String name) {
        this.name = name;
    }

    public void fly() {
        System.out.print("The Bird (" + this.name + ") can fly.");
    }
}

class Ostrich extends Bird {

    public Ostrich() {
        super("Ostrich");
    }

    public void runFast() {
        System.out.print("The Bird (" + super.name + ") can run very fast.");
    }
}

/*
LSP - Better Version
*/
class Bird2 {
    protected String name;
    protected String origin;

    public Bird2(String name) {
        this.name = name;
    }

    public String getName() {
        System.out.println("The name of the Bird is: " + this.name);
        return this.name;
    }
}
```

```
class RunningBird extends Bird2 {

    public RunningBird(String name) {
        super(name);
    }

    public void run() {
        System.out.println("The Bird (" + super.name + ") is a running Bird.");
    };
}

class FlyingBird extends Bird2 {

    public FlyingBird(String name) {
        super(name);
    }

    public void fly() {
        System.out.println("The Bird (" + super.name + ") is a flying Bird.");
    }
}

/*
LSP - Version with Interface
*/
/*
LSP - Breaking - Example
*/
class RectangleLSP {
    protected int width;
    protected int height;

    public void setHeight(int height) {
        this.height = height;
    }

    public void setWidth(int width) {
         this.width = width;
    }

    public int getArea() {
        return this.width * this.height;
    }
}

class SquareLSP extends RectangleLSP {
    @Override
```

```
    public void setWidth(int width) {
        super.width = width;
        super.height = width;
    }

    @Override
    public void setHeight(int height) {
        super.width = height;
        super.height = height;
    }
}

/*
LSP - Valid Example
*/
interface IShapeLSP {
    int getAreaLSP();
}

class RectangleLSP2 implements IShapeLSP {
    protected int width;
    protected int height;

    public RectangleLSP2(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getAreaLSP() {
        return this.width * this.height;
    }
}

class SquareLSP2 extends RectangleLSP2 {

    public SquareLSP2(int side) {
        super(side, side);
    }
}

/*
Interface - Segregation - Principle (ISP)
*/
/*
Bad Example
*/
interface BirdISP {
```

```
    public String getName();
    public void fly();
    public void run();
}

class FalconISP implements BirdISP {

    @Override
    public String getName() {
        return "Falcon";
    }

    @Override
    public void fly() {
        System.out.println("I'am a flying bird!");
    }

    @Override
    public void run() {
    }
}

class OstrichISP implements BirdISP {

    @Override
    public String getName() {
        return "Ostrich";
    }

    @Override
    public void fly() {
    }

    @Override
    public void run() {
        System.out.println("I'am a running Bird!");
    }
}

/*
Right Example
*/
interface IBirdISP_2 {
    public String getName();
}

interface IFlyingBirdISP extends IBirdISP_2 {
    public void fly();
```

```java
}

interface IRunningBirdISP extends IBirdISP_2 {
    public void run();
}

class FalconISP_2 implements IFlyingBirdISP {

    @Override
    public void fly() {
        System.out.println("I'am a flying bird!");
    }

    @Override
    public String getName() {
        return "Falcon";
    }
}

class OstrichISP_2 implements IRunningBirdISP {

    @Override
    public void run() {
        System.out.println("I'am a running Bird!");
    }

    @Override
    public String getName() {
        return "Ostrich";
    }
}

/*
Dependency - Inversion - Principle (DIP)
*/
/*
Abstract Example - Width directly dependency
*/
class LowLevelComponent {
    public void doSomething() {
        System.out.println("The Low-Level Component is doing something!");
    }
}

class HighLevelComponent {
    public void run(LowLevelComponent l) {
        l.doSomething();
    }
```

```java
}

/*
Abstract Example - Widthout directly dependency
*/
interface IDoSomething {
    public void doSomething();
}

class LowLevelComponent_2 implements IDoSomething {

    @Override
    public void doSomething() {
        System.out.println("The Low-Level Component is doing something!");
    }
}

class HighLevelComponent_2 {
    public void run(IDoSomething i) {
        i.doSomething();
    }
}

/**
 *
 */
public class PrinciplesOfProgramming {

    static final ArrayList<IVehicle> listVehicles = new ArrayList<>();

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        System.out.println("----------------------------------------");
        System.out.println("Vehicle ");
        System.out.println("----------------------------------------");

        //Use first version of Vehicle
        Vehicles v = new Vehicles();
        v.registerVehicle();

        ArrayList<String> numberPlates = v.getNumberPlateByBrand("BMW");

        for (String numberPlate : numberPlates){
            System.out.println("BMW with number plate: " + numberPlate);
        }
```

```
        System.out.println("----------------------------------------");
        System.out.println("Better Version with Interface");
        System.out.print("----------------------------------------");

        //Use second version of vehicle
        registerVehicle();
        for (IVehicle vehicle : listVehicles) {
            if (vehicle.getBrand().equals("BMW")) {
                System.out.println();
                System.out.print("BMW with number plate: "
                        + vehicle.getNumberPlate());

                if (vehicle.getIsInUse()) {
                    System.out.print(" -> Is in Use");
                }
            }
        }

        System.out.println();
        System.out.println();
        System.out.println("----------------------------------------");
        System.out.println("KISS Principle");
        System.out.println("Swap function");
        System.out.println("----------------------------------------");

        //Swap function demonstration
        String[] swapValues = {"Apple", "Banana"};

        //Output: Apple - Banana
        System.out.println(swapValues[0] + " - " + swapValues[1]);

        //Output: Banana - Apple
        String[] swapedValues = swap(swapValues);
        System.out.println(swapedValues[0] + " - " + swapedValues[1]);

        System.out.println();
        System.out.println("----------------------------------------");
        System.out.println("Check and Swap function");
        System.out.println("----------------------------------------");

        //Showing the message "Swap value is null!".
        checkAndSwap(null);

        //Showing the message "Swap value has not the right number of values to swap!".
        checkAndSwap(new String[] {"Apple", "Banana", "Kiwi"});

        //Output: Banana - Apple
        swapedValues = checkAndSwap(swapValues);
```

```
        System.out.println(swapedValues[0] + " - " + swapedValues[1]);

        System.out.println();
        System.out.println("----------------------------------------");
        System.out.println("Open - Close - Principle - Bad Example");
        System.out.println("----------------------------------------");

        Square s1 = new Square();
        s1.line_x = 3;

        Square s2 = new Square();
        s2.line_x = 8;

        Square s3 = new Square();
        s3.line_x = 10;

        ArrayList<Square> listSquares = new ArrayList<>();
        ArrayList<Rectangle> listRectangles = new ArrayList<>();
        listSquares.add(s1);
        listSquares.add(s2);
        listSquares.add(s3);

        CalculateArea c = new CalculateArea();
        int areaSquare_1 = c.calculateAreaOfSquare(s1);
        int areaSquare_2 = c.calculateAreaOfSquare(s2);
        int areaSquare_3 = c.calculateAreaOfSquare(s3);
        int areaAllSquares = c.calculateAreaOfAllShapes(listSquares, listRectangles);

        System.out.println("Area of Square 1: "+ areaSquare_1);
        System.out.println("Area of Square 2: "+ areaSquare_2);
        System.out.println("Area of Square 3: "+ areaSquare_3);
        System.out.println("Area of all Squares: "+ areaAllSquares);

        Rectangle r1 = new Rectangle();
        r1.line_x = 2;
        r1.line_y = 3;

        Rectangle r2 = new Rectangle();
        r2.line_x = 10;
        r2.line_y = 20;

        Rectangle r3 = new Rectangle();
        r3.line_x = 5;
        r3.line_y = 4;

        listRectangles.add(r1);
        listRectangles.add(r2);
        listRectangles.add(r3);
```

```
int areaRectangle_1 = c.calculateAreaOfRectangle(r1);
int areaRectangle_2 = c.calculateAreaOfRectangle(r2);
int areaRectangle_3 = c.calculateAreaOfRectangle(r3);
int areaOfAllShapes = c.calculateAreaOfAllShapes(listSquares, listRectangles);

System.out.println();
System.out.println("Area of Rectangle 1: "+ areaRectangle_1);
System.out.println("Area of Rectangle 2: "+ areaRectangle_2);
System.out.println("Area of Rectangle 3: "+ areaRectangle_3);
System.out.println("Area of all Shapes: "+ areaOfAllShapes);

System.out.println();
System.out.println("---------------------------------------");
System.out.println("Open - Close - Principle - Good Example");
System.out.println("---------------------------------------");

Square_2 square_1 = new Square_2(3);
Square_2 square_2 = new Square_2(8);
Square_2 square_3 = new Square_2(10);

Rectangle_2 rect_1 = new Rectangle_2(2, 3);
Rectangle_2 rect_2 = new Rectangle_2(10, 20);
Rectangle_2 rect_3 = new Rectangle_2(5, 4);

ArrayList<IShape> listShapes = new ArrayList<>();
listShapes.add(square_1);
listShapes.add(square_2);
listShapes.add(square_3);
listShapes.add(rect_1);
listShapes.add(rect_2);
listShapes.add(rect_3);

CalculateArea_2 calcArea = new CalculateArea_2();
int resSquare_1 = calcArea.calculateShape(square_1);
int resSquare_2 = calcArea.calculateShape(square_2);
int resSquare_3 = calcArea.calculateShape(square_3);
int resRect_1 = calcArea.calculateShape(rect_1);
int resRect_2 = calcArea.calculateShape(rect_2);
int resRect_3 = calcArea.calculateShape(rect_3);
int resAllShapes = calcArea.calculateAreaOfAllShapes(listShapes);

System.out.println("Area of Square 1: "+ resSquare_1);
System.out.println("Area of Square 2: "+ resSquare_2);
System.out.println("Area of Square 3: "+ resSquare_3);
System.out.println();
System.out.println("Area of Rectangle 1: "+ resRect_1);
System.out.println("Area of Rectangle 2: "+ resRect_2);
```

```
        System.out.println("Area of Rectangle 3: "+ resRect_3);
        System.out.println("Area of all Shapes: "+ resAllShapes);

        System.out.println();
        System.out.println("-----------------------------------------------");
        System.out.println("Open - Close - Principle - Add new Shape Circle");
        System.out.println("-----------------------------------------------");

        Circle circle = new Circle(10);
        int resCircle = calcArea.calculateShape(circle);
        listShapes.add(circle);
        resAllShapes = calcArea.calculateAreaOfAllShapes(listShapes);

        System.out.println("Area of Circle: "+ resCircle);
        System.out.println("Area of all Shapes (incl. Circle): "+ resAllShapes);

        System.out.println();
        System.out.println("-----------------------------------------------");
        System.out.println("The Liskov - Substitution - Principle (LSP)");
        System.out.println("-----------------------------------------------");

        System.out.println();
        System.out.println("Breaking the LSP");

        Bird falcon = new Bird("Falcon");
        Ostrich paul = new Ostrich();

        falcon.fly();
        System.out.println(" -- That's correct.");
        paul.fly();
        System.out.println(" -- That's wrong.");

        System.out.println();
        System.out.println("Correct LSP");

        Bird2 sparrow = new Bird2("Sparrow");
        FlyingBird falcon2 = new FlyingBird("Falcon");
        RunningBird ostrich = new RunningBird("Ostrich");

        sparrow.getName(); // Output: “Sparrow”
        falcon2.fly(); // that’s correct.
        ostrich.run(); // that’s correct.
        falcon2.getName(); // that’s correct
        ostrich.getName(); // that’s correct

        System.out.println();
        System.out.println("LSP - Example with Interface");
        System.out.println("LSP - Breaking - Example");
```

```
        System.out.println();

        System.out.println("Compute Rectangle with width = 2 and height = 5");
        RectangleLSP r = new RectangleLSP();
        SquareLSP s = new SquareLSP();

        printAreaLSP(r); // As expected.
        System.out.println(" -- Result as expected");

        printAreaLSP(s); // Not as ecpected.
        System.out.println(" -- Result not as expected");
        System.out.println();

        System.out.println("LSP - Valied - Example");
        System.out.println();

        RectangleLSP2 rLSP = new RectangleLSP2(2, 5);
        SquareLSP2 sLSP = new SquareLSP2(5);

        printAreaLSP2(rLSP); // Expect 10 and get out 10. Valid Result.
        System.out.println(" -- Expect 10 and get out 10. Valid Result.");

        printAreaLSP2(sLSP); // Expect 25 and get out 25. Valid Result.
        System.out.println(" -- Expect 25 and get out 25. Valid Result.");

        System.out.println();
        System.out.println("-------------------------------------------");
        System.out.println("The Interface - Segregation - Principle (ISP)");
        System.out.println("-------------------------------------------");

        System.out.println();

        FalconISP_2 falconISP = new FalconISP_2();
        OstrichISP_2 ostrichISP = new OstrichISP_2();

        System.out.println("I'am a " + falconISP.getName());
        falconISP.fly();

        System.out.println();
        System.out.println("I'am a " + ostrichISP.getName());
        ostrichISP.run();

        System.out.println();
    }

    private static void registerVehicle() {
        listVehicles.add(new Vehicle("Mercedes", "12AB34", false));
        listVehicles.add(new Vehicle("BMW", "43BA21", true));
```

```
        listVehicles.add(new Vehicle("BMW", "45RE67", false));
        listVehicles.add(new Vehicle("Volvo", "34tr56", true));
    }

    private static String[] swap(String[] values) {
        String[] ret = {values[1], values[0]};

        return ret;
    }

    private static String[] checkAndSwap(String[] values) {
        if (null == values) {
            System.out.println("Swap value is null!");
            return null;
        } else if (values.length != 2) {
            System.out.println("Swap value has not the right number of values to swap!");
            return null;
        } else {
            return swap(values);
        }
    }

    private static void printAreaLSP(RectangleLSP rectangle) {
        rectangle.setHeight(2);
        rectangle.setWidth(5);

        System.out.print("The area of the Rectangle is: " + rectangle.getArea());
    }

    private static void printAreaLSP2(IShapeLSP shape) {
        System.out.print("The area of the Rectangle is: " + shape.getAreaLSP());
    }
}
```